

Customer Churn Prediction & LTV Engine

Week 3 • Day 1

Customer Lifetime Value (LTV) — Concepts & Calculation

Calculate historical LTV and build the foundation for predictive LTV modelling

■ Day Objectives

- ◆ Understand Customer Lifetime Value (LTV) and why it matters to businesses
- ◆ Calculate historical LTV from TotalCharges and tenure
- ◆ Segment customers into LTV tiers (High, Medium, Low Value)
- ◆ Analyse the relationship between LTV and churn probability
- ◆ Save LTV segments to the database for dashboard use

■ 1. What is Customer Lifetime Value?

Customer Lifetime Value (LTV or CLV) is the total revenue a business expects to receive from a customer throughout the entire relationship. It answers: **'How much is this customer worth to us?'** Businesses use LTV to decide how much to spend on retention — you would spend more to keep a \$5,000 LTV customer than a \$200 one.

LTV Calculation Formula

- **Historical LTV:** Total revenue already received = TotalCharges
- **Simple Predictive LTV:** MonthlyCharges × Expected Remaining Tenure

- **Advanced Predictive LTV:** $\text{MonthlyCharges} \times (1/\text{churn_probability}) \times \text{discount_factor}$

■ ■ **Note:** For our project, we combine both: historical LTV from TotalCharges and predictive LTV using our churn model probabilities to estimate remaining months.

■ ■ 2. LTV Calculation Script

■ **src/models/ltv_model.py – LTV calculation and modelling**

```
import pandas as pd
import numpy as np
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, r2_score
import matplotlib.pyplot as plt
import seaborn as sns
import joblib, sys
sys.path.insert(0, '.')
from src.db.connection import get_engine
from sqlalchemy import text

ANNUAL_DISCOUNT_RATE = 0.10 # 10% annual discount rate

def calculate_historical_ltv(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    # Historical LTV = total revenue received to date
    df['historical_ltv'] = df['totalcharges']
    return df

def calculate_predictive_ltv(df, churn_probs):
    """
    Predictive LTV = MonthlyCharges * expected remaining months
    Expected remaining months ≈ 1 / churn_probability (geometric distribution)
    Apply discount factor for time value of money
    """
    df = df.copy()
    df['churn_probability'] = churn_probs

    # Avoid division by zero (cap very low probabilities)
    df['churn_probability'] = df['churn_probability'].clip(lower=0.01)

    # Expected remaining tenure (months) = 1 / monthly churn rate
    df['expected_remaining_months'] = 1.0 / df['churn_probability']
    df['expected_remaining_months'] = df['expected_remaining_months'].clip(upper=72) #
    cap at 6 years

    # Monthly discount factor
    monthly_discount = ANNUAL_DISCOUNT_RATE / 12

    # Discounted predictive LTV (Present Value of annuity)
    # PV = PMT * [1 - (1+r)^(-n)] / r
    n = df['expected_remaining_months']
    r = monthly_discount
    df['predictive_ltv'] = df['monthlycharges'] * (1 - (1+r)**(-n)) / r
```

```

df['total_ltv'] = df['historical_ltv'] + df['predictive_ltv']

return df

def segment_by_ltv(df):
    """Assign LTV tier based on total_ltv percentiles."""
    p33 = df['total_ltv'].quantile(0.33)
    p67 = df['total_ltv'].quantile(0.67)
    def tier(v):
        if v >= p67: return 'High Value'
        elif v >= p33: return 'Medium Value'
        else: return 'Low Value'
    df['ltv_segment'] = df['total_ltv'].apply(tier)
    return df, p33, p67

if __name__ == '__main__':
    # Load clean data
    df = pd.read_csv('data/processed/telco_features.csv')

    # Get churn probabilities from production model
    model = joblib.load('models/production_model.pkl')
    from src.features.preprocessing import preprocess_for_ml
    X_train, X_test, y_train, y_test, _, _, _ = preprocess_for_ml(df)

    # Get probabilities for ALL customers
    import warnings; warnings.filterwarnings('ignore')
    from src.features.preprocessing import preprocess_for_ml
    df_proc = df.copy()
    # Preprocess for scoring (use all data)
    df_temp = pd.get_dummies(df_proc.drop(columns=['customerid', 'churn', 'loyalty_segment'], errors='ignore'),
                               columns=['internetservice', 'contract', 'paymentmethod', 'multiplelines'],
                               drop_first=False)

    # Align columns
    feature_cols = joblib.load('models/feature_columns.pkl')
    for col in feature_cols:
        if col not in df_temp.columns:
            df_temp[col] = 0
    df_temp = df_temp[feature_cols]
    churn_probs = model.predict_proba(df_temp)[: , 1]

    # Calculate LTV
    df_ltv = calculate_historical_ltv(df_proc)
    df_ltv = calculate_predictive_ltv(df_ltv, churn_probs)
    df_ltv, p33, p67 = segment_by_ltv(df_ltv)

    print('\n=== LTV Summary ===')
    print(df_ltv[['total_ltv', 'ltv_segment', 'churn_probability']].describe())
    print('\nLTV Segment Distribution:')
    print(df_ltv['ltv_segment'].value_counts())

    # Save to CSV and DB
    df_ltv.to_csv('data/processed/telco_ltv.csv', index=False)

```

```
engine = get_engine()
df_ltv.to_sql('customers_ltv', con=engine, if_exists='replace', index=False)
print('Saved to data/processed/telco_ltv.csv and customers_ltv table')
```

■ 3. Commit & Checklist

- LTV calculated for all 7,043 customers
- Three LTV tiers created (High/Medium/Low Value)
- customers_ltv table saved to PostgreSQL

■ Week3-Day1 Commit

```
git add .
git commit -m "Week3-Day1: LTV calculation – historical + predictive LTV, customer segmentation"
git push origin main
```

■ Estimated Time: 5 – 6 hours

— End of Week 3 Day 1 — Zaalima Development Confidential —